

TOBB Ekonomi ve Teknoloji Üniversitesi

Bilgisayar Mühendisliği Bölümü

BİL 481 - Yazılım Mühendisliği

2025 Güz Dönemi

Proje Ödevi 3 (Deployment Plan)

NOTLA

Üniversite Ders Notları Paylaşım ve Değerlendirme Platformu

Abdullah Arda Gündoğdu 231401029

Ömer Talha Akbulut 221401005

Yunus Emre Özçelik 221401001

İsmail Emre Yıldız 221401006

Teslim Tarihi:

03.12.2025

Görev Alanı (Task Area)	Alt Görevler (Sub-tasks)	Sorumlu Kişi (Contributor)
Deployment Overview	Genel dağıtım mimarisinin (Render/Vercel yapısı) tanımlanması ve mimari yaklaşımın özetlenmesi.	Yunus Emre Özçelik
	Kullanılan araçların ve ortamların (PaaS, DB, CDN) teknik özelliklerinin belirlenmesi.	İsmail Emre Yıldız
Deployment Process	Backend (Render) tarafındaki adım adım kurulum sürecinin ve build scriptlerinin dokümanite edilmesi.	Abdullah Arda Gündoğdu
	Frontend (Vercel) tarafındaki entegrasyon adımlarının ve build ayarlarının yazılması.	Yunus Emre Özçelik
Configuration Plan	Ortam değişkenlerinin (Environment Variables) ve settings.py ayarlarının listelenmesi.	Ömer Talha Akbulut
	CORS, WhiteNoise ve API URL yapılandırmalarının detaylandırılması.	İsmail Emre Yıldız
Review & Finalization	Dokümanın genel akışının düzenlenmesi, dil kontrolü ve nihai hale getirilmesi.	Tüm Ekip

İçindekiler

1. DEPLOYMENT OVERVIEW (DAĞITIM GENEL BAKIŞ)

1.1. Mimari Yaklaşım (Client-Server Architecture) 1.2. Backend Ortamı: Render.com 1.3. Frontend Ortamı: Vercel 1.4. Sürüm Kontrolü ve CI/CD

2. DEPLOYMENT PROCESS (DAĞITIM SÜRECİ)

2.1. Aşama 1: Hazırlık ve Konfigürasyon Kontrolü 2.2. Aşama 2: Backend Dağıtımı (Render.com Kurulumu) 2.3. Aşama 3: Frontend Dağıtımı (Vercel Kurulumu)

3. CONFIGURATION PLAN (KONFIGÜRASYON PLANI)

3.1. Backend Konfigürasyonları (Environment Variables) 3.2. Kod İçi Kritik Ayarlar (Settings.py & WhiteNoise) 3.3. Frontend Konfigürasyonları (API Base URL) 3.4. Token Yönetimi (Interceptor & Service Headers) 3.5. Demo Kullanıcıları (Test Hesapları)

1. Deployment Overview (Dağıtım Genel Bakış)

Notla projesi, modern web standartlarına uygun olarak **İstemci-Sunucu (Client-Server)** mimarisiyle geliştirilmiş ve dağıtılmıştır. Demo sunumu için performans, ölçeklenebilirlik ve dağıtım kolaylığı sağlayan bulut tabanlı PaaS çözümleri tercih edilmiştir.

- **Mimari Yaklaşım:** Decoupled Architecture (Ayrık Mimari). Backend ve Frontend ayrı sunucularda barındırılır ve API üzerinden haberleşir.
- **Backend Ortamı: Render.com.**
 - Django projesi, gunicorn uygulama sunucusu üzerinde çalıştırılmıştır.
 - Veritabanı olarak Render tarafından sağlanan **PostgreSQL** kullanılmıştır.
 - Statik dosyalar (Admin paneli CSS/JS), WhiteNoise kütüphanesi ile doğrudan backend sunucusu üzerinden servis edilmektedir.
- **Frontend Ortamı: Vercel.**
 - React uygulaması, Vercel'in global CDN (Content Delivery Network) altyapısı üzerinde statik bir site olarak barındırılmaktadır.
- **Sürüm Kontrolü:** GitHub. Her iki platform da (Render ve Vercel), GitHub reposuna yapılan "push" işlemiyle tetiklenen **CI/CD (Sürekli Entegrasyon/Dağıtım)** boru hattını kullanır.

2. Deployment Process (Dağıtım Süreci)

Projenin canlıya alınması sürecinde aşağıdaki adım adım prosedür izlenmiştir:

Aşama 1: Hazırlık ve Konfigürasyon Kontrolü

1. **Backend Hazırlığı:**
 - Proje kök dizinine build.sh scripti eklendi. Bu script sırasıyla; bağımlılıkları yükler (pip install), statik dosyaları toplar (collectstatic) ve veritabanı migrasyonlarını (migrate) uygular.
 - settings.py dosyasında DEBUG modu, ortam değişkenine (RENDER) bağlandı ve ALLOWED_HOSTS ayarlandı.
 - CORS ayarları, frontend'in barınacağı domain'i kabul edecek şekilde yapılandırıldı.
2. **Frontend Hazırlığı:**
 - api.js içerisindeki localhost adresi, canlı ortamdaki backend API adresiyle değiştirilecek şekilde parametrik hale getirildi (Environment Variable kullanımı için hazırlandı).

Aşama 2: Backend Dağıtımı (Render.com)

1. **Web Service Oluşturma:** Render üzerinde yeni bir "Web Service" oluşturuldu ve GitHub reposundaki backend klasörü bağlandı.
2. **Runtime Ayarları:**
 - **Environment:** Python 3
 - **Build Command:** ./build.sh (Kullanıcının yazdığı script)
 - **Start Command:** gunicorn notla_backend.wsgi:application

3. **Veritabanı Bağlantısı:** Render üzerinde bir PostgreSQL veritabanı oluşturuldu ve DATABASE_URL otomatik olarak backend ortamına tanımlandı (dj_database_url bu bağlantıyı settings.py içinde okur).
4. **Deploy:** İlk deploy işlemi başlatıldı. build.sh çalışarak veritabanı tablolarını oluşturdu ve admin süper kullanıcıyı script içerisindeki komutla otomatik olarak yarattı.

Aşama 3: Frontend Dağıtımı (Vercel)

1. **Proje İçe Aktarma:** Vercel paneli üzerinden GitHub reposu seçildi ve frontend klasörü (src dizininin bulunduğu kök) "Root Directory" olarak ayarlandı.
2. **Build Ayarları:**
 - **Framework Preset:** Create React App
 - **Build Command:** npm run build
3. **API Bağlantısı:** Backend'in Render URL'i (örn: <https://notla-backend.onrender.com/api>), Vercel ortam değişkenlerine tanımlandı.

3. Configuration Plan (Konfigürasyon Planı)

Demo ortamının sorunsuz çalışması için uygulanan nihai konfigürasyonlar şunlardır:

A. Backend Konfigürasyonları (Environment Variables)

Render tarafında tanımlanan kritik değişkenler:

Değişken	Değer / Açıklama
PYTHON_VERSION	3.9.0 (Veya uygun sürüm)
SECRET_KEY	Üretim ortamı için oluşturulmuş rastgele, karmaşık hash string'i.
DEBUG	False (Güvenlik gereği kapalı, ancak RENDER değişkeni var olduğu için kod otomatik algılar).
DATABASE_URL	postgres://user:pass@host/dbname (PostgreSQL bağlantı adresi).

CORS_ALLOWED_ORIGINS	https://notla.vercel.app (Frontend'in erişimine izin verir).
----------------------	--

B. Kod İçi Kritik Ayarlar (settings.py)

- **WhiteNoise:** Statik dosyaların sıkıştırılarak sunulması için `whitenoise.storage.CompressedManifestStaticFilesStorage` aktif edildi.
- **Admin Kullanıcısı:** `build.sh` dosyası içerisindeki script sayesinde, deploy anında veritabanında admin / 123456 kullanıcısı yoksa otomatik oluşturulması sağlandı.

C. Frontend Konfigürasyonları

Frontend tarafında API isteklerinin doğru yere gitmesi için yapılan düzenleme

API Base URL: `frontend/src/api.js` dosyasındaki sabit localhost tanımı yerine, dinamik yapı tercih edilmelidir (veya demo için manuel güncellenmiştir)

Token Yönetimi: Uygulamada kimlik doğrulama token'larının yönetimi için hibrit (karma) bir yapı kullanılmıştır:

- **Global Interceptor (api.js):** Axios instance'ı, her HTTP isteğinden önce çalışacak bir interceptor ile yapılandırılmıştır. Bu yapı, `localStorage` üzerinde `notlaToken` anahtarı altında bir veri bulursa, bunu otomatik olarak `Authorization: Bearer` başlığına eklemektedir.
- **Manuel Header Yönetimi (courseService.js):** Ders notları ve yorum işlemleri gibi spesifik servis çağrılarında, global interceptor'dan bağımsız olarak çalışan manuel bir başlık ekleme mekanizması bulunmaktadır. Bu servis fonksiyonları, `getAuthHeaders` fonksiyonunu çağırarak token'ı `access_token` anahtarı üzerinden okumaya ve isteğe eklemeye çalışacak şekilde yapılandırılmıştır.

D. Demo Kullanıcıları

Sunum sırasında kullanılacak hazır hesaplar:

1. **Admin:** `admin@etu.edu.tr` / 123456 (Yönetim paneli ve tüm yetkiler)
2. **Öğrenci:** `ogrenci@etu.edu.tr` / 123456 (Not yükleme ve yorum yapma senaryoları için)